

CS324 Distributed Systems

Assignment One



Team Members

Name	ID
Aryan Singh	S11230987
Shivan Prasad	S11231502
Mohammed Suhail	S11230995
Praheel Kumar	S11229535

Theory Question Solutions

Question 1: Explain the importance of Bootstrap Node in the system. How could the system be implemented without using a Bootstrap Node?

The Bootstrap Node provide an initial discovery point for workers joining the distributed system. The workers form an unstructured network where each worker is connected to only a subset of the other workers. Therefore, whenever a new worker starts, it needs a way to find atleast one preexisting worker before it can become a part of it. The assignment requires the Bootstrap Node to run through as a separate java process and communicate through java RMI and remain separate from leader election and computational job processing.

In our implementation, the worker registers its ID, host and port with the Bootstrap Node and requests for an initial neighbor. If it is the first worker, no neighbor is returned. Otherwise, the Bootstrap Node selects one eligible registered worker and returns its connection information, then the joining worker connects directly to that worker using java RMI and the neighbor relationship is established in both directions. The Bootstrap service also provides deregistration operation so that a worker's registration can be removed when required.

The Bootstrap Node is particularly useful because a new worker joining does not need to know the addresses of all existing workers beforehand. It only needs to know one fixed location to join the system.

The system could also operate without a Bootstrap Node by using another discovery method. For example, seed nodes where each worker could be configured with the address of one or more known workers, a worker joining could contact a seed worker to get information about other peers. Another option would be peer to peer discovery mechanism where workers exchange neighbor information. These approaches remove Bootstrap Node, but they make the initial discovery and configuration more complicated.

Question 2: Does the inclusion of Bootstrap Node make the Overall system centralized?

The Bootstrap Node does not make the entire system centralized; however, it introduces a centralized component for registration and initial worker discovery.

Whenever a worker joins, it contacts the Bootstrap Node to register and obtain an initial neighbor. This means the joining process has a central point. However, once the worker has joined, normal conversation takes place directly between the workers using java RMI. The Bootstrap Node does not relay on normal worker messages, participate in leader elections or process computational jobs as the assignment required it to stay out of it.

A useful way to describe this design is that it has a centralized discovery but distributed operations. The Bootstrap Node is a central dependency for new workers joining, so if it becomes unavailable, new worker may be not able to register and discover their initial neighbors, however it is not completely responsible for coordinating all communication or computation after workers have joined.

Thus, Bootstrap Node only centralizes the registration and discovery function and the entire distributed system.

Question 3: Duplicate Message Prevention and its importance

In our implementation, duplicate election messages are prevented by assigning a unique message ID to every ELECTION message. The ID is generated when the election is started and is included whenever the message is forwarded between workers. Each worker maintains a collection of election message ID's that it has already processed.

When a worker receives an ELECTION message, it first checks whether the message ID already exists in its collection. If the ID is not present, the worker records the ID, processes the election message and forwards it to its neighbouring workers. If the same message reaches the worker again through another neighbour, the worker detects that the message ID has already been processed and ignores the duplicate.

This mechanism is particularly important because the workers form an unstructured network. A worker can have multiple paths to another worker, meaning that the same election message can reach a worker through different neighbours. Without duplicate detection, the same election could be processed multiple times, causing unnecessary network traffic and repeated election processing. It could also lead to incorrect or inconsistent election results if workers repeatedly update their election state.

For example, if Worker 1 sends an ELECTION message to both Worker 2 and Worker, and both workers forward the message to Worker 4, Worker 4 may receive same election twice. Since both messages contain the same unique message ID, Worker 4 processes the first message and ignores the second one.

The same approach can also be applied to COORDINATOR messages. Each coordinator announcement has a unique message ID, allowing workers to update their coordinator information once and prevent repeated propagation of the same announcement.

Overall, using unique message IDs and a processed message collection allows the election to be safely propagated through the unstructured network while avoiding duplicate processing and unnecessary message flooding.

Question 4: Explain how threads were used in your implementation to support concurrent processing. What thread-safety mechanisms were implemented to protect shared data and/or prevent race conditions and/or deadlocks? Provide examples from your system.

Threads are used at three points in our implementation. On the client side, each ClientGuiFrame submits jobs through a dedicated thread pool instead of the Swing event thread so the GUI stays responsive and a second job can be submitted before the first returns. On the coordinator side, the elected worker partitions each job into sub-tasks and dispatches them to workers concurrently through its own thread pool, collecting the resulting futures and blocking only to aggregate the final result so computation across workers happens in parallel rather than sequentially. On the worker side, each worker uses a fixed-size thread pool to execute its assigned sub-tasks, allowing it to process more than one at a time rather than serialising them.

Since these pools all operate on data shared across threads neighbour lists, election state and job counters several thread-safety mechanisms were required. Shared collections such as each worker's neighbour list, processed election message IDs and pending election replies use concurrent collection types so RMI calls arriving from several neighbours at once during a flooding election can safely read and update this state without external locking. Counters read and updated from multiple threads each worker's Job Allocation Counter, the coordinator's persistent JAC and its per-term job count use atomic integer types so an increment is treated as one indivisible step even when two clients submit jobs at effectively the same instant. The coordinator's job submission method is also synchronized, preventing two threads from interleaving the term-expiry check, job processing and counter update.

```
private final ExecutorService clientSubmissionPool = Executors.newCachedThreadPool();

private void handleAsyncJobSubmission() {
    // Dispatches computation in a background thread so the GUI remains interactive
    clientSubmissionPool.submit(() -> {
        coordinatorService coordinator = lookupCoordinator(host, port);
        JobResult result = coordinator.submitJob(request);
        // Safely updates the GUI on the Event Dispatch Thread
        SwingUtilities.invokeLater(() -> statusLabel.setText("Completed " + jobId));
    });
}
```

```
private final ExecutorService dispatchPool = Executors.newCachedThreadPool();

// Submit all subtasks in parallel
List<Future<JobResult>> futures = new ArrayList<>();
for (int i = 0; i < tasks.size(); i++) {
    JobTask task = tasks.get(i);
    WorkerService worker = activeWorkers.get(i % numWorkers);

    futures.add(dispatchPool.submit(() -> worker.executeJob(task)));
}

// Gather results in parallel
for (Future<JobResult> future : futures) {
    subResults.add(future.get()); // waits only as long as the longest subtask
}
```

Git Commit Logs

Git History X				
Enter to search commit		Repo: CS324-Assignment...	Branch: All	Author
Graph	Message	Date	Author	Hash
	add random peer discovery	2026-09-22 00:05	Siddharat	f16baa6
	implement bootstrap server startup	2026-09-21 22:43	Siddharat	cf5113
	implement worker registration	2026-09-21 22:02	Siddharat	3222bec
	workload-computation-&l-partitioning origin origin/message-handling Merge...	2026-09-20 12:36	Aryan Singh	a9f6306
	Merge pull request #1 from aryann-7/worker-side-multithreading	2026-09-20 12:36	Aryan Singh	85863bb
	feat(node): add concurrent node coordination logic	2026-09-20 12:29	Your Name	5d6762c
	feat(system): add concurrent job execution	2026-09-20 12:27	Your Name	776d514
	feat(system): add system-side workload partitioning	2026-09-20 12:26	Your Name	082cb77
	worker-side-multithreading origin feat(worker): add worker-side multithreading ...	2026-09-20 11:46	Your Name	1618587
	feat(scripts): initial cluster startup scripts	2026-09-18 10:32	Your Name	4614e13
	feat(data): initial sample benchmark CSV datasets (starting point without functionality)	2026-09-18 10:31	Your Name	813a510
	feat(client): initial skeleton for GUI, CSV parser, and client submission (starting point wi...	2026-09-18 10:31	Your Name	923c533
	feat(worker): initial skeleton for worker node, election, runner, and coordinator (startin...	2026-09-18 10:31	Your Name	584dad0
	feat(bootstrap): initial skeleton for bootstrap server and membership service (starting ...	2026-09-18 10:29	Your Name	16ff311
	feat(common): initial skeleton for RMI remote interfaces and models (starting point wit...	2026-09-18 10:28	Your Name	7acb2d4
	Initial commit	2026-09-17 21:23	Aryan Singh	8424b3d

Git History X				
Enter to search commit		Repo: CS324-Assignment...	Branch: All	Author
Graph	Message	Date	Author	Hash
	Merge pull request #6 from aryann-7/suhail-section2	2026-09-23 22:04	Mohammed Suhail	f7cae39
	Improve worker election flooding	2026-09-23 21:52	Mohammed Suhail	97ff453
	Merge pull request #5 from aryann-7/suhail-section2	2026-09-23 21:38	Mohammed Suhail	82a6456
	Implement election state and duplicate prevention	2026-09-23 21:35	Mohammed Suhail	00f63a2
	Merge pull request #4 from aryann-7/resolve-merge-conflicts	2026-09-23 14:36	Aryan Singh	37a6f49
	resolve-merge-conflicts origin chore(fix): resolve merge conflicts	2026-09-23 14:31	Your Name	134a244
	chore: resolve merge conflict in bootstrap implementation	2026-09-23 14:15	Your Name	32db96d
	chore: add project .gitignore	2026-09-23 14:08	Your Name	1e3520e
	Merge pull request #3 from aryann-7/feature/bootstrap-registry	2026-09-23 13:40	Aryan Singh	0ac71f4
	origin/feature/bootstrap-registry Merge branch 'main' into feature/bootstrap-regi...	2026-09-23 13:38	Aryan Singh	30c1b44
	origin/shivan/bootstrap-task5-safe add bootstrap registry tests	2026-09-23 00:00	Siddharat	e5b1fea
	Sectio 2	2026-09-22 23:44	Mohammed Suhail	5665ae0
	Implement election message handling	2026-09-22 23:33	Mohammed Suhail	7a32008
	Implement election message handling	2026-09-22 23:04	Mohammed Suhail	f0516f4
	implement worker deregistration	2026-09-22 22:19	Siddharat	15a130b
	add random peer discovery	2026-09-22 00:05	Siddharat	f16baa6

Git History X

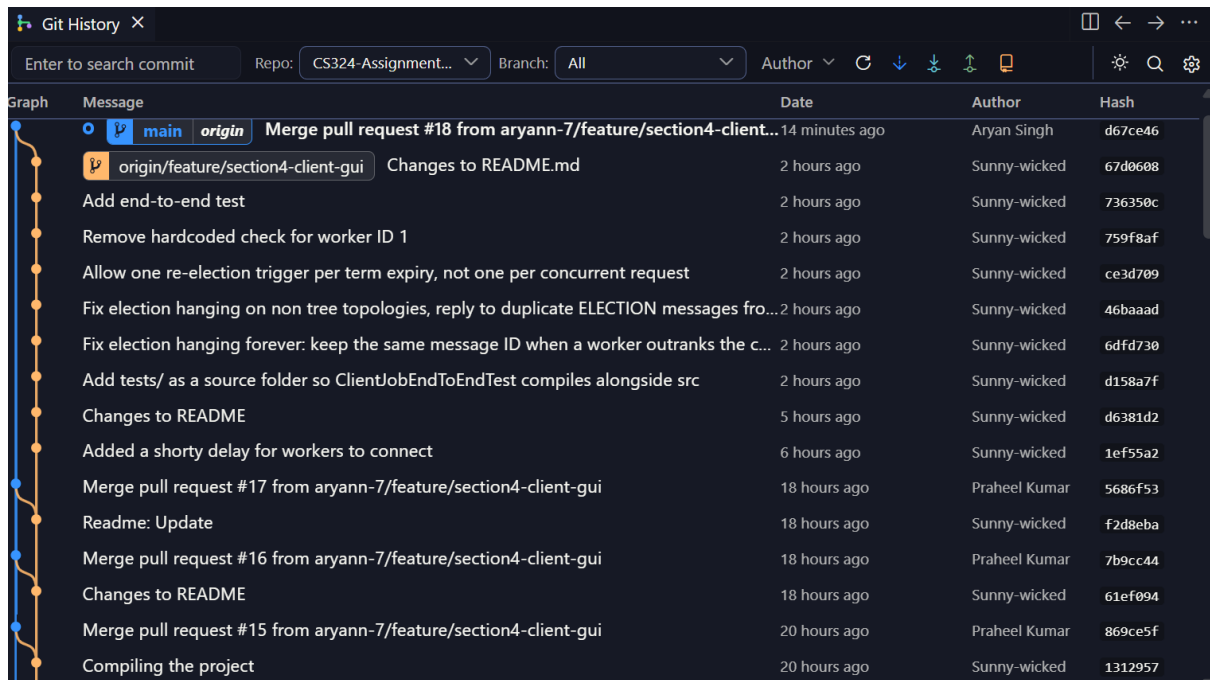
Enter to search commit Repo: CS324-Assignment... Branch: All Author

Graph	Message	Date	Author	Hash
	origin/shivan/bootstrap-task6-integration add worker bootstrap integration tests	2026-09-24 02:37	Siddharat	aae7239
	add bidirectional neighbour registration	2026-09-24 00:41	Siddharat	8746656
	Merge pull request #11 from aryann-7/suhail-section2	2026-09-24 00:38	Mohammed Suhail	9087996
	align bootstrap discovery contract	2026-09-24 00:24	Siddharat	d3b7de7
	origin/suhail-section2 Register WorkerNode.stepDownToWorker as the coordinat...	2026-09-24 00:20	Mohammed Suhail	b7c9cfb
	Bind coordinator to RMI registry on election so clients can locate it	2026-09-24 00:03	Mohammed Suhail	6b7b4a8
	Merge pull request #10 from aryann-7/suhail-section2	2026-09-23 23:54	Mohammed Suhail	0e3dd3d
	Register WorkerNode.stepDownToWorker as the coordinator's term-expiry listener	2026-09-23 23:52	Mohammed Suhail	55623db
	Merge pull request #9 from aryann-7/suhail-section2	2026-09-23 23:47	Mohammed Suhail	307eaa6
	Trigger re-election immediately after the 5th job assignment ends the term	2026-09-23 23:46	Mohammed Suhail	6a92cd6
	Trigger re-election when a job is rejected due to expired term	2026-09-23 23:44	Mohammed Suhail	727a8e7
	Merge pull request #8 from aryann-7/suhail-section2	2026-09-23 23:41	Mohammed Suhail	962d896
	Add notifyTermExpired helper to fire re-election on a separate thread	2026-09-23 23:29	Mohammed Suhail	403b55f
	Merge pull request #7 from aryann-7/suhail-section2	2026-09-23 23:24	Mohammed Suhail	c6d8a5f
	Add notifyTermExpired helper to fire re-election on a separate thread	2026-09-23 23:14	Mohammed Suhail	6480391
	Merge pull request #6 from aryann-7/suhail-section2	2026-09-23 22:04	Mohammed Suhail	f7cae39

Git History X

Enter to search commit Repo: CS324-Assignment... Branch: All Author

Graph	Message	Date	Author	Hash
	Compiling the project	20 hours ago	Sunny-wicked	1312957
	Setting up the client GUI	2026-09-26 19:04	Sunny-wicked	d324e0e
	Setting up the client GUI	2026-09-26 19:04	Sunny-wicked	66ba5d9
	Vscode set up	2026-09-26 09:40	Sunny-wicked	49d252f
	Merge pull request #13 from aryann-7/process-refine	2026-09-25 15:50	Aryan Singh	c6fdd33
	process-refine origin feat(scripts): add add_worker convenience scripts for dyna...	2026-09-25 15:48	Your Name	346be49
	feat(coordinator): discover and partition jobs across all cluster workers	2026-09-25 15:46	Your Name	dec1ebd
	test(worker & bootstrap): update integration & unit tests for multi-neighbor mesh con...	2026-09-25 14:14	Your Name	aef5470
	feat(bootstrap): enable multi-neighbor discovery for worker network resilience	2026-09-25 14:14	Your Name	f5440e8
	feat(scripts): add instance number passing for GUI title bar	2026-09-25 13:58	Your Name	9be283a
	feat: add CSV dataset parsing logic	2026-09-25 13:57	Your Name	77fb875
	feat: add client dynamic coordinator lookup & asynchronous execution	2026-09-25 13:56	Your Name	b252ef6
	feat: add dynamic coordinator discovery	2026-09-25 13:52	Your Name	7c59eb6
	feat(scripts): add dynamic scripts for multiple workers and clients	2026-09-25 11:26	Your Name	106b195
	Merge pull request #12 from aryann-7/shivan/bootstrap-task6-integration	2026-09-24 07:16	Aryan Singh	e1f9003
	origin/shivan/bootstrap-task6-integration add worker bootstrap integration tests	2026-09-24 02:37	Siddharat	aae7239



Git Author Mapping

Git Author Name	Account Name	Student ID
Aryan Singh	aryann-7	S11230987
Mohammed Suhail	MohammedSuhail9080	S11230995
Praheel Kumar	Sunny-wicked	S11229535
Shivan Prasad	Shivan Siddharat	S11231502

Member Contribution Table

Member Name	Student ID	Design Roles	Implementation Roles	Testing Roles	Contribution
Aryan Singh	S11230987	Workload partitioning & compute task interfaces	Computational tasks (MAX, PRIMESUM, PRIMECOUNT) & coordinator worker dispatch	Concurrency & sub-task partition unit tests	25%
Shivan Prasad	S11231502	Custom leader election & term-management protocols	ELECTION message flooding, JAC tie-breaking, coordinator announcement.	Election race condition & message duplicate loop tests	25%
Mohammed Suhail	S11230995	Bootstrap node & unstructured peer discovery design	Bootstrap RMI registry, node registration, dynamic random neighbor linking	Node bootstrap & bidirectional connection integration tests	25%
Praheel Kumar	S11229535	Client GUI architecture & file I/O workflows	Client desktop GUI, CSV parsing, asynchronous RMI submission	Multi-client end-to-end integration tests	25%

Team Meeting Details: Project Kickoff & Architectural Partitioning

Date & Time: 17 September 2026, 3:00 PM – 4:30 PM

Location: Library

Attendees: Aryan Singh, Shivan Prasad, Mohammed Suhail, Praheel Kumar

Agenda:

- Deconstruct assignment requirements, deliverables, and RMI platform constraints.
- Agree on repository setup and branching strategy.
- Divide initial system responsibilities across Bootstrap, Election, Computation, and Client GUI.

Key Decisions:

- Use a single Git repository (CS324-Assignment) with feature branches corresponding to each assignment section.
- Shivan to lead the Bootstrap discovery module, Suhail to lead the custom leader election, Aryan to build the computational engine, and Praheel to build the Client GUI.
- Sockets/HTTP ruled out; enforce strict Java RMI registry standards throughout.